

SKILL_WEEK - 08 (SESSION NUMBER - 15)

Task 1: To parse combined redirection, duplicate file descriptors, merge streams, verify ordering, handle edge cases, and test integrated outputs.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main()
{
    int fd;
    int saved_stdout;
    int saved_stderr;

    saved_stdout = dup(STDOUT_FILENO);
    saved_stderr = dup(STDERR_FILENO);

    fd = open("combined_output.txt",
              O_WRONLY | O_CREAT | O_TRUNC,
              0644);

    if (fd < 0)
    {
        perror("File open error");
        return 1;
    }

    dup2(fd, STDOUT_FILENO);
    dup2(fd, STDERR_FILENO);

    close(fd);

    printf("This is standard output.\n");
    fprintf(stderr, "This is standard error.\n");

    fflush(stdout);
    fflush(stderr);

    dup2(saved_stdout, STDOUT_FILENO);
    dup2(saved_stderr, STDERR_FILENO);

    close(saved_stdout);
    close(saved_stderr);

    printf("Streams restored successfully.\n");

    return 0;
}
```

OUTPUT:

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/session15_task1
Streams restored successfully.
[amrutha@Amruthas-MacBook-Air OSSP % cat combined_output.txt
This is standard error.
This is standard output.
amrutha@Amruthas-MacBook-Air OSSP %
```

Task 2: To parse complex commands, combine pipes and redirections, resolve execution order, validate syntax, generate execution plans, and test scenarios.

```
    dup2(input, STDIN_FILENO);
    dup2(fd[1], STDOUT_FILENO);

    close(input);
    close(fd[0]);
    close(fd[1]);

    execlp("cat", "cat", NULL);

    perror("cat failed");
    exit(1);
}

p2 = fork();

if (p2 == 0)
{
    int output = open("complex_output.txt",
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);

    if (output < 0)
    {
        perror("Output file error");
        exit(1);
    }

    dup2(fd[0], STDIN_FILENO);
    dup2(output, STDOUT_FILENO);

    close(output);
    close(fd[0]);
    close(fd[1]);

    execlp("grep", "grep", "Operating", NULL);

    perror("grep failed");
    exit(1);
}

close(fd[0]);
close(fd[1]);

waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);

printf("Complex pipe and redirection executed successfully.\n");

return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % echo "Operating Systems is interesting" > complex_input.txt
echo "Linux shell programming" >> complex_input.txt
echo "Operating Systems practical" >> complex_input.txt
amrutha@Amruthas-MacBook-Air OSSP % gcc src/session15_task2.c -o bin/session15_task2
./bin/session15_task2
Complex pipe and redirection executed successfully.
amrutha@Amruthas-MacBook-Air OSSP % cat complex_output.txt
Operating Systems is interesting
Operating Systems practical
amrutha@Amruthas-MacBook-Air OSSP %
```

(SESSION NUMBER - 16)

Task 1: To detect background jobs, launch non-blocking processes, return the prompt immediately, store job information, monitor execution, and test background tasks.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Background job started.\n");
        printf("Child PID: %d\n", getpid());

        sleep(5);

        printf("Background job completed.\n");
        exit(0);
    }
    else
    {
        printf("Background process launched.\n");
        printf("Stored Job PID: %d\n", pid);
        printf("Prompt returned immediately.\n");

        waitpid(pid, NULL, 0);

        printf("Background job monitored successfully.\n");
    }

    return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/session16_task1.c -o bin/session16_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/session16_task1
Background process launched.
Stored Job PID: 2579
Prompt returned immediately.
Background job started.
Child PID: 2579
Background job completed.
Background job monitored successfully.
amrutha@Amruthas-MacBook-Air OSSP %
```

Task 2: To create a job table, store process groups, track states, update status, remove completed jobs, and validate job records.

```
UW PICO 5.09

    }
    else
    {
        jobs[i].job_id = i + 1;
        jobs[i].pid = pid;
        jobs[i].pgid = pid;
        sprintf(jobs[i].state, "RUNNING");
    }
}

printf("Job Table:\n");

for (int i = 0; i < job_count; i++)
{
    printf("Job %d | PID: %d | PGID: %d | State: %s\n",
        jobs[i].job_id,
        jobs[i].pid,
        jobs[i].pgid,
        jobs[i].state);
}

for (int i = 0; i < job_count; i++)
{
    waitpid(jobs[i].pid, NULL, 0);
    sprintf(jobs[i].state, "COMPLETED");

    printf("Job %d status updated to COMPLETED.\n",
        jobs[i].job_id);
}

printf("\nRemoving completed jobs...\n");

job_count = 0;

printf("Job table is now empty.\n");

return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/session16_task2.c -o bin/session16_task2
[./bin/session16_task2
Job Table:
Job 1 | PID: 3465 | PGID: 3465 | State: RUNNING
Job 2 | PID: 3466 | PGID: 3466 | State: RUNNING
Job 1 status updated to COMPLETED.
Job 2 status updated to COMPLETED.

Removing completed jobs...
Job table is now empty.
amrutha@Amruthas-MacBook-Air OSSP %
```